=== UNIT 1: PROGRAMMING CONCEPTS, C TOKENS, DATA TYPES & OPERATORS ===

>> 1.1 Program Development Cycle and C Compilation Pipeline

C is a structured, procedural, statically-typed programming language developed by Dennis Ritchie at Bell Labs (1972).
* Four-Stage Compilation Pipeline:
1. Preprocessing (cpp): Strips comments, expands preprocessor directives (#include, #define), executes conditional compilation (#ifdef). Generates expanded source file (.i).
2. Compilation (c1): Translates preprocessed source code into target architecture Assembly code (.s), performing lexical analysis, syntax analysis (AST), semantic checks, and code optimization.
3. Assembly (as): Converts assembly instructions into machine code object files containing relocatable binary (.o or .obj).
4. Linking (ld): Resolves cross-file symbol references and links external library functions (e.g., libc.a, libm.a) to produce the final executable binary (.exe or a.out).

>> 1.2 C Tokens, Literals and Primitive Data Types

* C Tokens: Smallest individual units in a program: Keywords (32 standard ANSI keywords), Identifiers (user-defined names starting with letter or underscore), Constants, String Literals, and Special Symbols.
* Primitive Data Types and Typical Memory Footprints:
- char: 1 byte (8 bits, range -128 to +127 or 0 to 255 for unsigned char). Format specifier: %c.
- short int: 2 bytes (16 bits, range -32,768 to +32,767). Format specifier: %hd.
- int: 4 bytes (32 bits, range -2,147,483,648 to +2,147,483,647). Format specifier: %d.
- float: 4 bytes (IEEE-754 single precision, 6-7 decimal digits precision). Format specifier: %f.
- double: 8 bytes (IEEE-754 double precision, 15-17 decimal digits precision). Format specifier: %lf.

>> 1.3 Operators, Precedence and Type Conversions

* Arithmetic Operators: Addition (+), Subtraction (-), Multiplication (*), Division (/, truncates integer quotients), Modulo (%, remainder of integer division only).
* Relational and Logical Operators: <, <=, >, >=, ==, !=. Logical AND (&&), OR (||), NOT (!). Exhibit short-circuit evaluation: in (A && B), if A is false, B is never evaluated.
* Bitwise Operators: Bitwise AND (&), OR (|), XOR (^), NOT (~), Left Shift (<<,

multiplies by 2^n), Right Shift (>>, divides by 2^n).
* Ternary Conditional Operator: (condition) ? expression_if_true : expression_if_false.
* Precedence and Associativity: Parentheses () have highest precedence. Unary operators (!, ++, --, sizeof) associate Right-to-Left. Multiplicative (*, /, %) associate Left-to-Right. Assignment (=, +=, -=) associates Right-to-Left.
* Type Casting: Implicit (Automatic type promotion following hierarchy: char -> int -> float -> double) vs. Explicit (programmer-directed coercion: (type)expression).

[PROGRAM 1: Complete Temperature Conversion and Precision Formatting]

Problem: Write an ANSI C program to convert temperature from Fahrenheit to Celsius using formula C = (5.0/9.0) * (F - 32). Demonstrate integer vs. floating-point division pitfalls and format specifiers.
Source Code:

```
#include <stdio.h>
int main() {
double fahrenheit, celsius;
printf("Enter temperature in Fahrenheit: ");
if (scanf("%lf", &fahrenheit) != 1) {
printf("Invalid numeric input!\n");
return 1;
}
/* Note: Writing 5/9 causes integer division truncation to 0. Must use 5.0/9.0 */
celsius = (5.0 / 9.0) * (fahrenheit - 32.0);
printf("-------------------------------------------------\n");
printf("Fahrenheit: %10.2f deg F\n", fahrenheit);
printf("Celsius: %10.2f deg C\n", celsius);
printf("Kelvin: %10.2f K\n", celsius + 273.15);
printf("-------------------------------------------------\n");
return 0;
}
```

Program Trace for Input 98.6 deg F (Human Body Temperature):
Step 1: fahrenheit = 98.6.
Step 2: fahrenheit - 32.0 = 66.6.
Step 3: (5.0 / 9.0) = 0.5555556.
Step 4: celsius = 0.5555556 * 66.6 = 37.00 deg C.
Output: Celsius = 37.00 deg C | Kelvin = 310.15 K.

[PROGRAM 2: Bitwise Manipulation and Endianness Detection]

Problem: Write a program to: (1) Check if processor architecture is Little-Endian or

Big-Endian using pointers and bitwise access, (2) Count total set bits (1s) in a 32-bit integer using Brian Kernighan's algorithm.

Source Code:

```
#include <stdio.h>
int countSetBits(unsigned int n) {
int count = 0;
while (n > 0) {
n = n & (n - 1); /* Clears lowest set bit in O(k) iterations */
count++;
}
return count;
}
int main() {
unsigned int x = 1;
char *c = (char*)&x;
if (*c == 1)
printf("Architecture: Little-Endian (LSB stored first)\n");
else
printf("Architecture: Big-Endian (MSB stored first)\n");
unsigned int val = 29;
printf("Integer: %u | Set Bits Count: %d\n", val, countSetBits(val));
return 0;
}
```

Algorithm Trace for val = 29 (Binary 11101):

Iteration 1: n = 29 (11101) & 28 (11100) = 28. count = 1.

Iteration 2: n = 28 (11100) & 27 (11011) = 24. count = 2.

Iteration 3: n = 24 (11000) & 23 (10111) = 16. count = 3.

Iteration 4: n = 16 (10000) & 15 (01111) = 0. count = 4. Terminate.

Output: Architecture: Little-Endian | Set Bits Count: 4.

[PROGRAM 3: Operator Precedence and Short-Circuit Evaluation Demonstration]

Problem: Write a program to verify short-circuit evaluation of logical operators && and || in C expressions containing embedded side-effect increments.

Source Code:

```
#include <stdio.h>
int main() {
int a = 0, b = 5, c = 10;
int res1 = (a && (++b));
printf("res1: %d | a: %d | b: %d (b was NOT incremented)\n", res1, a, b);
int res2 = (b || (++c));
printf("res2: %d | b: %d | c: %d (c was NOT incremented)\n", res2, b, c);
int res3 = (a || (++b));
printf("res3: %d | a: %d | b: %d (b WAS incremented to 6)\n", res3, a, b);
```

```
return 0;
}
```

Dry-Run Output:

res1: 0 | a: 0 | b: 5 (b was NOT incremented)

res2: 1 | b: 5 | c: 10 (c was NOT incremented)

res3: 1 | a: 0 | b: 6 (b WAS incremented to 6)

Conclusion: Logical expressions must never embed required variable modifications because short-circuit optimization bypasses right-hand operands.

[PROGRAM 4: Quadratic Equation Root Solver with Discriminant Analysis]

Problem: Write a robust C program to compute real and complex roots of a quadratic equation ax^2 + bx + c = 0.

Source Code:

```
#include <stdio.h>
#include <math.h>
int main() {
double a, b, c, disc, root1, root2, realPart, imagPart;
printf("Enter coefficients a, b, c: ");
scanf("%lf %lf %lf", &a, &b, &c);
if (a == 0.0) {
printf("Coefficient ’a’ cannot be zero in a quadratic equation.\n");
return 1;
}
disc = b * b - 4.0 * a * c;
if (disc > 0.0) {
root1 = (-b + sqrt(disc)) / (2.0 * a);
root2 = (-b - sqrt(disc)) / (2.0 * a);
printf("Roots are real and distinct: Root1 = %.4f, Root2 = %.4f\n", root1, root2);
} else if (disc == 0.0) {
root1 = -b / (2.0 * a);
printf("Roots are real and equal: Root1 = Root2 = %.4f\n", root1);
} else {
realPart = -b / (2.0 * a);
imagPart = sqrt(-disc) / (2.0 * a);
printf("Roots are complex conjugates:\n");
printf("Root1 = %.4f + %.4fi\n", realPart, imagPart);
printf("Root2 = %.4f - %.4fi\n", realPart, imagPart);
}
return 0;
}
```

>> 1.4 University Review & Examination Questions

* Q1: Explain the four stages of the C compilation system in detail, naming the intermediate files produced.
Answer: The compilation system transforms source code (prog.c) into machine executable through four stages: (1) Preprocessing (cpp): Executes all directives beginning with '#'. Replaces #include header lines with literal text of header files, replaces #define macros with replacement strings, and strips comments. Produces expanded preprocessed source file (prog.i). (2) Compilation (c1): Analyzes syntax, builds abstract syntax trees, and optimizes code, translating high-level C code into target-specific assembly code (prog.s). (3) Assembly (as): Translates assembly mnemonics into machine code instructions, packaging them into relocatable object format (prog.o). (4) Linking (ld): Resolves external references, connects runtime startup code, and links standard C library functions (e.g., printf in libc), generating final executable binary (prog.exe or a.out).
* Q2: Differentiate between implicit type conversion (type promotion) and explicit type casting in C with examples.
Answer: Implicit conversion is performed automatically by the compiler when operands of mixed types appear in an expression. The smaller type is promoted following standard conversion hierarchy: char/short -> int -> unsigned int -> long -> float -> double -> long double. Example: in 'int i = 5; float f = 2.5; float res = i + f;', i is promoted to float (5.0f). Explicit casting is performed intentionally by the programmer using '(target_type)expression'. Example: 'int a = 5, b = 2; double d = (double)a / b;' explicitly casts 'a' to double (5.0), forcing floating-point division to yield 2.5 instead of integer truncation 2.

>> 1.5 Key Takeaways and Revision Points

* C compilation consists of Preprocessor -> Compiler -> Assembler -> Linker.
* Integer division in C truncates remainders; floating-point literals require decimal notation (e.g., 5.0 / 9.0).
* Logical AND (&&) and OR (||) exhibit short-circuit evaluation, skipping right-hand operands when outcome is determined.
* Bitwise manipulation operates directly on binary representation (&, |, ^, ~, <<, >>).